Correção · Algoritmos · Árvores · treino (15 questões)

Data: 11/11/2026 · **Semana 3** · **Placar: 12/15 (80%)** · 3 erros · 0 "não sabia"

Respostas lidas das anotações do PDF: 1-E · 2-D · 3-C · 4-B · 5-E · 6-A · 7-C · 8-A · 9-A · 10-B · 11-E · 12-C · 13-B · 14-E · 15-A

> Questões autorais no estilo FGV, elaboradas para treino. Não são questões oficiais.
> **Como li as marcas:** usei os círculos nas letras (a tabela "Minhas respostas" ficou vazia). **Nenhum "?"**. Um **pontinho** ao lado da (B) da Q10, que conta como resposta com dúvida. Os ✓ ao lado das alternativas (Q3, Q5, Q7, Q9) e no item I da Q8 foram lidos como "verdadeiro". Você desenhou a árvore em **todas** as questões com chaves inseridas (Q2, Q3, Q5, Q6, Q7, Q11).

Questão	Assunto	Sua	Gabarito	Resultado
1	Folhas, nós internos e grau (n-ária)	E	B	❌
2	Caminho da busca na BST	D	D	✅
3	Busca de chave ausente + ponto de inserção	C	C	✅
4	Busca na BST em Java (código)	B	B	✅
5	Remoção com sucessor em ordem	E	E	✅
6	Remoção com antecessor em ordem	A	A	✅
7	Remoção (1 filho e folha) + pré-ordem	C	C	✅
8	Maior da BST, folhas, n − 1 arestas (I/II/III)	A	D	❌
9	Recursão em Java: conta nós internos	A	A	✅
10	Recursão em Python: altura com caso base −1	B (pontinho)	D	❌
11	Folhas e altura de BST montada	E	E	✅
12	Percurso em pós-ordem	C	C	✅
13	AVL: nó que viola a condição	B	B	✅
14	BST degenerada (chaves decrescentes)	E	E	✅
15	Altura mínima e máxima com 20 nós	A	A	✅

Por tipo:

Tipo	Questões	Diagnóstico (10/11)	Treino (hoje)
Busca na BST (caminho, chave ausente, código)	2, 3, 4	0/1	**3/3**
Remoção (sucessor, antecessor, 1 filho, folha)	5, 6, 7	0/1	**3/3**
Recursão em código	9, 10	0/1	1/2
Contagem e propriedades (folhas, arestas, altura)	1, 8, 11, 15	1/3	2/4
Revisão: percursos, AVL, degeneração	12, 13, 14	4/4	**3/3**

Acertos firmes: 12 de 12. **Erros:** 2 convictos (Q1 e Q8) e 1 com pontinho (Q10).

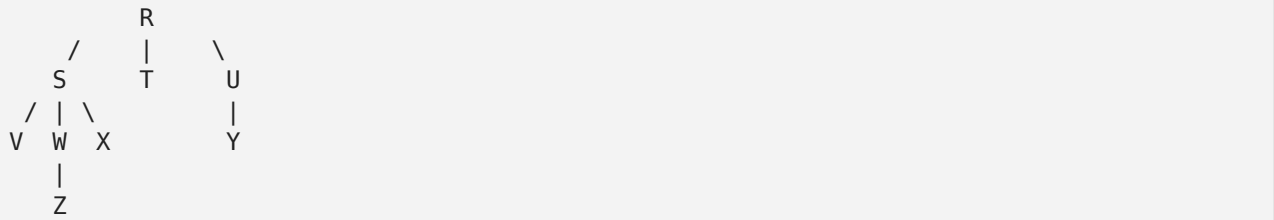
Comparação: árvores saiu de **5/10 (50%)** no diagnóstico para **12/15 (80%)** no treino. Os dois pontos que mais pesaram ontem estão resolvidos: **buscar (3/3) e remover (3/3)**, inclusive o sucessor e o antecessor com o filho que sobe. O hábito de **desenhar a árvore** funcionou: você desenhou nas 6

questões com chaves inseridas e acertou as 6. Os 3 erros de hoje são todos de **contagem e propriedade**, não de operação: a folha no nível 1 (o mesmo erro do diagnóstico), o "n − 1 arestas" em árvore n-ária e a altura em arestas × nós.

Árvores sai da fila (teoria mínima + diagnóstico + treino corrigido).

Questão 1 · ❌ Errou

O sistema e-LOTAÇÃO representa a hierarquia de unidades de uma comarca na árvore a seguir, em que R é a raiz:



O número de folhas, o número de nós internos e o grau da árvore são, respectivamente:

(A) 4, 5 e 3;

✅ **(B) 5, 4 e 3;**

(C) 1, 8 e 3;

(D) 5, 4 e 2;

❌ (E) 4, 4 e 3. ← sua resposta

Onde você se confundiu: você sublinhou **V, X, Y e Z** como folhas e deixou o **T** de fora. O T está no nível 1, logo abaixo da raiz, e **não tem filho nenhum**: é folha. É **o mesmo erro do diagnóstico** (lá foi o F, também no nível 1). A procura de folhas está indo direto para "o pé" do desenho e pulando o meio.

A conferência que salvava: a (E) diz 4 folhas + 4 internos = **8 nós**, mas a árvore tem **9** (R, S, T, U, V, W, X, Y, Z). A alternativa se contradiz sozinha.

Contagem certa: folhas = V, Z, X, **T**, Y (5) · internos = R, S, W, U (4) · grau = 3 (R e S têm 3 filhos).

Por que as outras erram: (A) conta o T como nó interno (5 internos) · (C) só conta o nó mais fundo (Z) · (D) dá grau 2 (o grau da árvore é o maior: 3).

📌 **Folha = nó sem filhos, em qualquer nível.** Leia os nós **um a um, na ordem do desenho** (R, S, T, U, V...) e pergunte "tem perninha para baixo?". Antes de marcar: **folhas + internos = total de nós**.

Questão 2 · ✅ Acertou

O sistema e-MANDADOS guarda os números dos mandados em uma árvore binária de busca, sem balanceamento, montada com a inserção, nesta ordem, das chaves 52, 30, 75, 18, 41, 60, 88, 35, 47 e 44. O analista Murilo localizou a chave 44 utilizando o algoritmo de busca próprio da árvore binária de busca.

A sequência de nós visitados, da raiz até encontrar a chave, é:

(A) 52, 30, 18, 41, 35, 47, 44;

(B) 52, 30, 75, 18, 41, 60, 88, 35, 47, 44;

(C) 52, 30, 41, 35, 47, 44;

✅ **(D) 52, 30, 41, 47, 44;** ← sua resposta

(E) 18, 30, 35, 41, 44.

Por quê: você desenhou a árvore e circulou em azul só o caminho 52 → 30 → 41 → 47 → 44. Um nó por nível, um lado só. (A) e (C) metem um irmão no caminho (18 é irmão do 41; 35 é irmão do 47) · (B) é a ordem de inserção (= percurso em nível) · (E) começa pelo menor, não pela raiz.

📌 Busca na BST = **um caminho só, da raiz para baixo**. Se aparecem dois irmãos, não é busca.

Questão 3 · ✅ Acertou

No sistema e-PERÍCIAS, uma árvore binária de busca, sem balanceamento, foi montada com a inserção, nesta ordem, das chaves 70, 40, 90, 20, 55, 80, 95, 50 e 60. A analista Lívia buscou a chave 58, que não está na árvore, pelo algoritmo de busca da árvore binária de busca.

O número de nós comparados até a busca concluir que a chave não existe e a posição em que a chave 58 seria inserida são, respectivamente:

(A) 9 nós, pois uma chave ausente obriga a percorrer a árvore inteira;

(B) 3 nós, e a chave seria inserida como filho direito do 55;

✅ **(C) 4 nós, e a chave seria inserida como filho esquerdo do 60;** ← sua resposta

(D) 4 nós, e a chave seria inserida como filho direito do 60;

(E) 5 nós, e a chave seria inserida como filho esquerdo do 50.

Por quê: 70 → 40 → 55 → 60 (4 comparações). 58 < 60, então desce para a esquerda do 60, que está vazia: a busca termina ali, e esse é exatamente o lugar onde o 58 entraria. (A) confunde com busca em lista · (B) para um nível antes (o filho direito do 55 já está ocupado pelo 60) · (D) vai para o lado errado · (E) o 50 nem está no caminho.

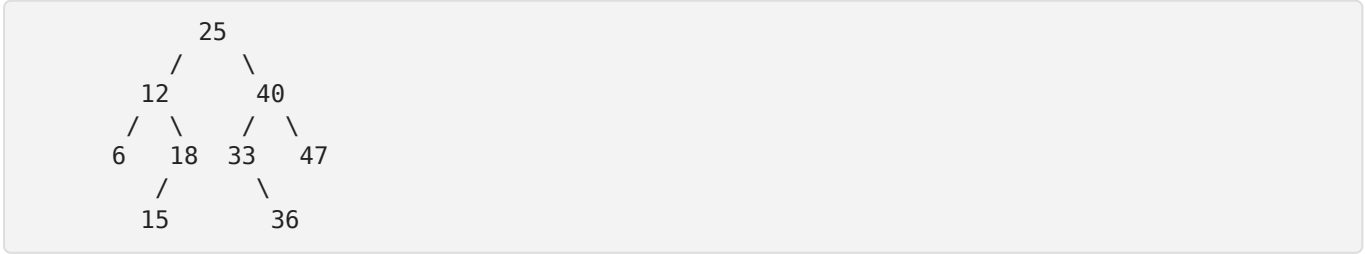
📌 Chave ausente: a busca para no primeiro filho vazio, e **esse filho vazio é a posição de inserção**.

Questão 4 · ✅ Acertou

O analista Otávio revisa o método a seguir, do sistema SISPAUTA, escrito em Java, que procura uma chave em uma árvore binária de busca:

```
No busca(No r, int x) {
    while (r != null && r.chave != x) {
        System.out.print(r.chave + " ");
        if (x < r.chave) r = r.esq;
        else r = r.dir;
    }
    return r;
}
```

O método foi chamado com a raiz da árvore a seguir e x = 36:



Ao final da execução, o método:

(A) imprime 25 40 33 36 e retorna o nó de chave 36;

✅ **(B) imprime 25 40 33 e retorna o nó de chave 36;** ← sua resposta

(C) imprime 25 12 40 33 e retorna o nó de chave 36;

(D) imprime 25 40 33 e retorna null;

(E) imprime 6 12 15 18 25 33 e retorna o nó de chave 36.

Por quê: você riscou o "36" da (A). Correto: quando `r` chega ao 36, a condição `r.chave != x` fica falsa e o laço **sai antes do print**. Então o 36 não é impresso, mas é retornado. (C) inclui o 12, que é o outro lado · (D) só retornaria null se a chave não existisse · (E) é o percurso em ordem.

📌 Em `while (cond) { print; anda }`, o nó que **encerra o laço não é impresso**. Rastreie a condição antes do print.

Questão 5 · ✅ Acertou

A árvore binária de busca do sistema e-DEPÓSITOS foi montada com a inserção, nesta ordem, das chaves 50, 25, 80, 10, 35, 65, 90, 60, 70 e 62. A analista Sílvia removeu a chave 50, que está na raiz e tem dois filhos, usando o **sucessor em ordem** para ocupar o lugar do nó removido.

Após a remoção, a nova raiz e a nova posição do nó 62 são, respectivamente:

(A) 65, e o 62 passa a ser filho esquerdo do 70;

(B) 80, e o 62 não muda de posição;

(C) 35, e o 62 não muda de posição;

(D) 62, e o 60 passa a ser filho direito do 65;

✅ **(E) 60, e o 62 passa a ser filho esquerdo do 65.** ← sua resposta

Por quê: você fez o caminho certo em azul: um passo à direita (80) e depois tudo à esquerda (65 → **60**). O 60 não tem filho esquerdo, então é o sucessor. Ele tinha o 62 como filho direito, e o 62 sobe para o lugar dele (filho esquerdo do 65). (A) parou no 65 sem descer tudo · (B) é o filho direito direto, **o erro do diagnóstico, agora evitado** · (C) 35 é o antecessor · (D) o 62 não é o menor da direita.

📌 Sucessor em ordem = **um passo à direita, depois tudo à esquerda**. Se ele tiver filho direito, esse filho sobe para o lugar dele.

Questão 6 · ✅ Acertou

No sistema e-ALVARÁS, a árvore binária de busca foi montada com a inserção, nesta ordem, das chaves 30, 15, 45, 8, 22, 40, 50, 19, 26 e 24. O analista Paulo removeu a chave 30, que está na raiz e tem dois filhos, usando desta vez o **antecessor em ordem** para ocupar o lugar do nó removido.

Após a remoção, a árvore fica assim:

✅ **(A) o 26 passa a ser a raiz, e o 24 passa a ser filho direito do 22;** ← sua resposta

(B) o 22 passa a ser a raiz, e o 26 passa a ser filho direito do 19;

(C) o 40 passa a ser a raiz, por ser o menor valor da subárvore direita;

(D) o 15 passa a ser a raiz, por ser o filho esquerdo da raiz removida;

(E) o 24 passa a ser a raiz, por ser a última chave inserida na subárvore esquerda.

Por quê: espelho da Q5. Um passo à esquerda (15) e depois tudo à direita (22 → **26**). O 26 não tem filho direito, então é o antecessor. O filho esquerdo dele (24) sobe para o lugar que ele ocupava, como filho

direito do 22. (B) parou no 22 · (C) é o sucessor, não o antecessor · (D) é o filho direto · (E) ordem de inserção não é critério.

📌 Antecessor em ordem = **um passo à esquerda, depois tudo à direita**. O filho esquerdo dele sobe.

Questão 7 · ✅ Acertou

A árvore binária de busca do sistema e-CARTAS foi montada com a inserção, nesta ordem, das chaves 44, 22, 66, 11, 33, 77 e 88. Em seguida, a analista Joana removeu a chave 77 e, depois, a chave 11.

O percurso em pré-ordem da árvore resultante é:

(A) 44, 22, 11, 33, 66, 88;

(B) 44, 22, 33, 66;

✅ **(C) 44, 22, 33, 66, 88;** ← sua resposta

(D) 22, 33, 44, 66, 88;

(E) 33, 22, 88, 66, 44.

Por quê: o 77 tem um filho só (88), que sobe para o lugar dele; o 11 é folha e só sai. Você até numerou a ordem da pré-ordem na árvore (1 a 5). (A) esqueceu de remover o 11 · (B) apagou a subárvore inteira do 77 (o 88 some) · (D) é o percurso em ordem · (E) é a pós-ordem.

📌 Remoção com **1 filho: o filho sobe**. Folha: só sai. Nada da subárvore se perde.

Questão 8 · ❌ Errou

Em uma revisão técnica do sistema e-INDEX, que usa árvores como índices em memória, a equipe de desenvolvimento de um Tribunal de Justiça analisou as afirmativas a seguir.

I. Em uma árvore binária de busca, o maior valor está no nó alcançado descendo sempre à direita a partir da raiz, e esse nó pode ter filho esquerdo.

II. Em uma árvore binária de busca, toda folha contém um valor menor que o valor da raiz.

III. Uma árvore com 12 nós tem exatamente 11 arestas, seja ela binária ou n-ária.

Está correto o que se afirma em

❌ (A) I, apenas. ← sua resposta

(B) II, apenas.

(C) III, apenas.

✅ **(D) I e III, apenas.**

(E) I, II e III.

Onde você se confundiu: I (✓) e II (X) você julgou certo, inclusive aplicando a regra do diagnóstico (o maior pode ter filho esquerdo). O erro foi o **X no III**. Provavelmente o "seja ela binária ou n-ária" fez parecer que uma árvore com mais filhos por nó teria mais arestas. Não tem: o número de **filhos** muda o formato da árvore, mas não a conta de arestas.

Por que n − 1 sempre: cada aresta liga um nó **ao seu pai**. Todo nó tem exatamente um pai, menos a raiz. Então 12 nós = 11 arestas, com qualquer número de filhos por nó.

```
Binária (12 nós)            N-ária (12 nós)
cada nó com até 2 filhos    raiz com 11 filhos
→ 11 arestas                → 11 arestas
```

No diagnóstico (Q10, item I) você aceitou "n nós = n − 1 arestas". Hoje, com "n-ária" no item, desconfiou. A palavra extra não muda a regra.

Item II: falso. As folhas da subárvore **direita** são maiores que a raiz.

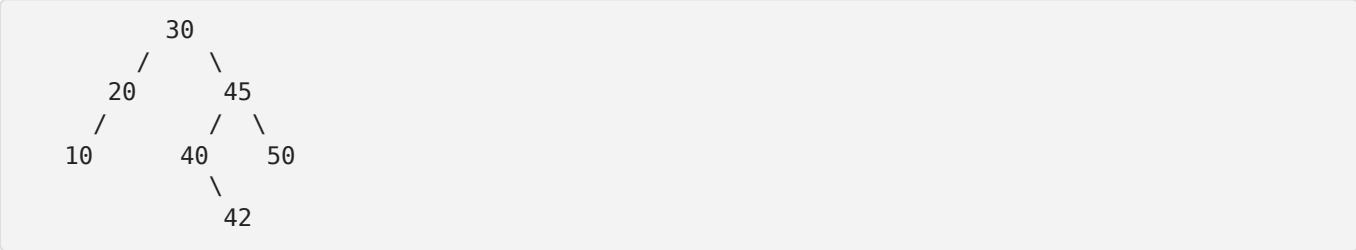
📌 **Toda árvore com n nós tem n − 1 arestas: binária, n-ária, BST ou AVL.** Cada nó, menos a raiz, tem uma aresta até o pai.

Questão 9 · ✅ Acertou

O analista Rodrigo encontrou o método recursivo a seguir no sistema SISCOR, escrito em Java:

```
int g(No n) {
    if (n == null) return 0;
    if (n.esq == null && n.dir == null) return 0;
    return 1 + g(n.esq) + g(n.dir);
}
```

O método foi chamado com a raiz da árvore binária a seguir:



O valor retornado e o que o método calcula são, respectivamente:

✅ **(A) 4, o número de nós internos (que não são folhas) da árvore;** ← sua resposta

(B) 7, o número de nós da árvore;

(C) 3, o número de folhas da árvore;

(D) 3, a altura da árvore;

(E) 2, o número de nós com dois filhos.

Por quê: você circulou 30, 20, 45 e 40 e riscou as folhas 10, 42 e 50. A folha devolve **0** e todo o resto soma 1, então conta quem **não** é folha. Rastreio: g(40) = 1 · g(45) = 1 + 1 + 0 = 2 · g(20) = 1 · g(30) = 1 + 1 + 2 = **4**. (B) contaria nós (sem a linha da folha) · (C) contaria folhas (folha devolvendo 1) · (D) altura usa `max` · (E) só 30 e 45 têm dois filhos, mas 20 e 40, com um filho, também somam.

📌 Recursão em árvore: **olhe o que a linha da folha devolve.** Folha → 0 e o resto soma 1 = conta nós internos.

Questão 10 · ❌ Errou (com pontinho)

A analista Helena encontrou a função recursiva a seguir no sistema e-JURIS, escrita em Python:

```
def h(no):
    if no is None:
        return -1
    return 1 + max(h(no.esq), h(no.dir))
```

A função foi chamada com a raiz da árvore binária a seguir:

```
      8
     / \
    3   10
   / \    \
  1   6    14
     / \   /
    4   7 13
```

O valor retornado e o que a função calcula são, respectivamente:

(A) 9, o número de nós da árvore;

❌ (B) 4, a altura da árvore, pois o caminho mais longo tem 4 nós; ← sua resposta (com pontinho)

(C) 4, o número de folhas da árvore;

✅ **(D) 3, a altura da árvore, contada em arestas;**

(E) 2, o nível do nó 6.

Onde você se confundiu: você circulou o `max` e viu certo que a função calcula a **altura**. Faltou fazer a conta com o **−1** do caso base. O caminho mais longo (8 → 3 → 6 → 4) tem mesmo 4 nós, mas a função não devolve 4.

Rastreio:

Chamada	Conta	Devolve
h(None)	caso base	−1
h(4), h(7), h(1), h(13)	1 + max(−1, −1)	**0** (folha = 0)
h(6)	1 + max(0, 0)	1
h(3)	1 + max(h(1)=0, h(6)=1)	2
h(14)	1 + max(h(13)=0, −1)	1
h(10)	1 + max(−1, h(14)=1)	2
h(8)	1 + max(2, 2)	**3**

O −1 serve justamente para a folha valer 0. Com folha = 0, a função conta **arestas**: 3. E o topo do bloco já avisava que altura = número de arestas.

Por que as outras erram: (A) seria contar nós (soma, não max) · (C) a árvore tem mesmo 4 folhas (1, 4, 7, 13), mas a função não conta folhas · (E) o 6 está no nível 2, mas a função é chamada na raiz.

📌 **Altura recursiva: `None → −1` dá altura em ARESTAS (folha = 0). `None → 0` dá o número de NÍVEIS (nós do caminho).** Rastreie a folha primeiro: o valor que ela devolve define a unidade.

Questão 11 · ✅ Acertou

O sistema e-SESSÕES mantém os códigos das sessões de julgamento em uma árvore binária de busca, sem balanceamento, montada com a inserção, nesta ordem, das chaves 40, 20, 60, 10, 30, 50, 70, 25 e 65.

O número de folhas e a altura da árvore resultante são, respectivamente:

(A) 2 e 3;

(B) 4 e 4;

(C) 5 e 3;

(D) 9 e 3;

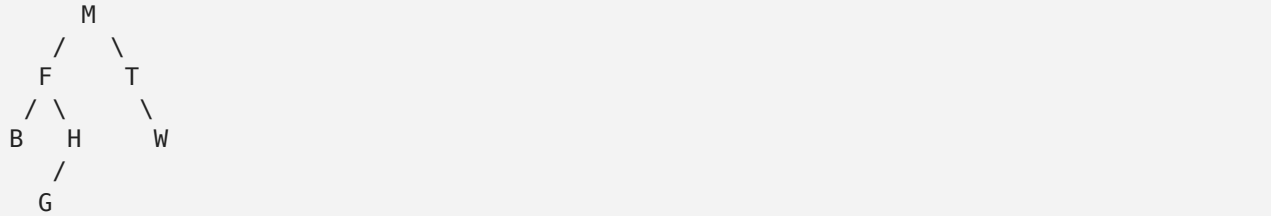
✅ **(E) 4 e 3.** ← sua resposta

Por quê: você desenhou a árvore, circulou as folhas 10, 25, 50 e 65 e escreveu "3" para a altura (40 → 20 → 30 → 25). Aqui o 30 e o 70 **não** são folhas (cada um tem um filho), e você não caiu nisso. (A) só os nós do último nível · (B) altura em nós · (C) conta o 30 ou o 70 como folha · (D) 9 é o número de nós.

📌 Nó com **um** filho não é folha. Altura = arestas do caminho mais longo.

Questão 12 · ✅ Acertou

O sistema e-NOTIFICA organiza as categorias de notificação na árvore binária a seguir:



O percurso em pós-ordem dessa árvore é:

(A) M, F, B, H, G, T, W;

(B) B, F, G, H, M, T, W;

✅ **(C) B, G, H, F, W, T, M;** ← sua resposta

(D) M, F, T, B, H, W, G;

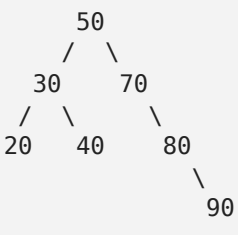
(E) W, T, M, H, G, F, B.

Por quê: você riscou a primeira letra da (A), da (D) e da (E): pós-ordem não começa pela raiz nem pelo nó mais à direita. Pós-ordem = esquerda, direita, raiz: a raiz M é a **última**. (A) pré-ordem · (B) em ordem · (D) em nível · (E) em ordem invertida.

📌 Pós-ordem: **raiz por último**. Pré-ordem: raiz primeiro.

Questão 13 · ✅ Acertou

O analista Vinícius verifica se a árvore binária de busca a seguir, do sistema e-AGENDA, atende à condição de uma árvore AVL. Considere que a altura de uma subárvore vazia é −1.



A condição AVL é violada no nó:

(A) 50, pois a subárvore direita tem mais nós que a subárvore esquerda;

✅ **(B) 70, pois a diferença entre as alturas de suas subárvores é 2;** ← sua resposta

(C) 80, pois ele tem apenas um filho;

(D) 90, pois é a folha mais profunda da árvore;

(E) nenhum, pois toda árvore binária de busca é balanceada.

Por quê: no 70, esquerda vazia (−1) e direita com altura 1 (80 → 90): diferença 2. No 50, esquerda 1 e direita 2: diferença 1, ok. (A) AVL compara **alturas**, não número de nós · (C) um filho só não viola nada (diferença 1) · (D) folha tem fator 0 · (E) BST sem balanceamento pode degenerar (Q14).

📌 AVL: **|altura esq − altura dir| ≤ 1 em todo nó**. Compara alturas, nunca quantidade de nós.

Questão 14 · ✅ Acertou

Na migração do sistema e-ARQUIVO, as chaves 100, 90, 80, 70, 60, 50, 40, 30, 20 e 10 foram inseridas, nessa ordem, em uma árvore binária de busca sem balanceamento. Em seguida, o analista Igor buscou a chave 10.

A altura da árvore e o número de nós comparados na busca são, respectivamente:

(A) 3 e 4, pois a árvore fica balanceada com 10 chaves;

(B) 9 e 10, pois a árvore degenera em uma lista que cresce sempre para a direita;

(C) 10 e 1, pois o menor valor fica na raiz;

(D) 1 e 10, pois todos os nós ficam no mesmo nível;

✅ **(E) 9 e 10, pois a árvore degenera em uma lista que cresce sempre para a esquerda.** ← sua resposta

Por quê: chaves em ordem **decrescente**: cada nova é menor que todas e vai sempre para a esquerda. 10 nós em fila = 9 arestas, e o 10 está no fim (10 comparações). Você riscou o "direita" da (B), que é a pegadinha (seria o caso de chaves crescentes). (A) só com balanceamento · (C) a raiz é o 100, a primeira inserida · (D) impossível numa árvore.

📌 BST sem balanceamento com chaves **ordenadas** vira lista: crescente → direita, decrescente → esquerda. Busca O(n).

Questão 15 · ✅ Acertou

A equipe da analista Clara vai armazenar 20 registros do sistema e-CONCILIA em uma árvore binária, com a raiz no nível 0.

A menor e a maior altura possíveis para essa árvore são, respectivamente:

✅ **(A) 4 e 19;** ← sua resposta

(B) 4 e 20;

(C) 5 e 19;

(D) 3 e 19;

(E) 10 e 19.

Por quê: você escreveu 2, 4, 8, 16 na margem (nós por nível). Níveis 0 a 3 cabem 1 + 2 + 4 + 8 = 15 < 20, então precisa do nível 4: altura mínima **4**. A máxima é a árvore degenerada (um nó por nível): 20 nós = **19** arestas. (B) 20 é o número de nós, não de arestas · (C) conta níveis (5), não altura · (D) com altura 3 cabem só 15 · (E) 10 é n/2, sem sentido aqui.

📌 Binária com n nós: **altura mínima = $\lfloor \log_2 n \rfloor$** (árvore cheia/completa) · **altura máxima = n − 1** (degenerada).

Para o caderno de erros

Assunto	Por que errei	Regra certa
Contar folhas (Q1)	Esqueci o T, no nível 1, **pela 2ª vez seguida** (no diagnóstico foi o F). Não conferi o total	Ler os nós um a um na ordem do desenho. Folhas + internos = total (a (E) dava 8 com 9 nós)
n − 1 arestas (Q8, item III)	O "n-ária" me fez achar que mudava a conta	Toda árvore com n nós tem n − 1 arestas: cada nó, menos a raiz, tem uma aresta até o pai
Altura recursiva (Q10)	Vi que era altura (circulei o `max`), mas contei nós (4) sem rastrear o −1	`None → −1`: folha = 0, altura em **arestas** (3). `None → 0`: em níveis

---

O que estudar agora

Quando	O que fazer
Agora, 10 min	Refazer **Q1** lendo os nós um a um e conferindo folhas + internos = 9 · **Q10** preenchendo a tabela de rastreio sem olhar
22:40 · revisão	Regras 📌 de hoje (árvores: as 3 dos erros) + revisão espaçada de 04/11 (valor-limite, partição, caixa-branca × preta, pirâmide) + 15 min de Anki
Qui 12/11 · almoço	🔴 **G. Semântica e reescrita** (1º da fila de Português), se ainda não começou
Qui 12/11 · 19h Bloco 1	🔴 **APF funções de dados:** responder o bloco de 06/11 (8 questões, ~20 min)
Qui 12/11 · 20h Bloco 2	🔴 **Hash:** diagnóstico pronto (8 questões). Não depende de árvores
Dom 15/11 · 13h	Simulado de domingo (rotina do plano)

Hábito confirmado: desenhar a árvore funcionou (6 de 6 nas questões com chaves inseridas). O próximo passo é o mesmo cuidado nas **contagens**: contar folha por folha e conferir o total, e rastrear a recursão começando pelo valor que a folha devolve.